

SatsPath Protocol v0.1

A Universal Bitcoin Payment Resolver and Router

SatsPath is a prototype protocol layer that lets users send Bitcoin using a single human-readable identifier while automatically selecting the best available payment rail.

Example identifiers:

```
rodrigo@satspath.dev  
alice@example.com  
julian@bitcoin.dev
```

The goal is not to replace Bitcoin, Lightning, Ark, BIP-353, LNURL, or BOLT12.

The goal is to build a resolution and routing layer above them.

```
Human-readable identifier  
  ↓  
Resolver  
  ↓  
Signed Payment Profile  
  ↓  
Routing Engine  
  ↓  
Lightning / On-chain / Ark  
  ↓  
Payment Execution or Payment Intent
```

1. Core Problem

Bitcoin already has multiple payment rails:

- On-chain Bitcoin
- Lightning Network
- LNURL / Lightning Address
- BOLT12
- Ark
- Silent Payments
- Future payment protocols

But normal users do not want to think about rails.

They want to do this:

```
Pay Rodrigo
Send 21,000 sats
Done
```

Today, users need to understand:

- Should I use an on-chain address?
- Are fees too high?
- Does the recipient support Lightning?
- Is the Lightning invoice expired?
- Does the recipient use Ark?
- Which option is cheapest?
- Which option is fastest?
- Which option preserves privacy?
- What happens if the receiver is not registered?

SatsPath solves this by separating the user experience from the payment rail.

The user provides one identifier. The protocol resolves the receiver's signed payment profile and chooses the best payment method.

2. Core Design Principle

Do not build another wallet first.

The first version of SatsPath should be:

```
identifier → signed profile → encode/decode → route selection → payment
simulation
```

A real wallet can be built later on top of the protocol.

For the hackathon, the most important deliverable is a working protocol prototype.

3. What SatsPath Is

SatsPath is:

- A payment identity resolver.
- A signed payment profile format.
- A routing algorithm for Bitcoin payment rails.
- A universal QR / URI format.
- A fallback system for unregistered users.
- A future foundation for split payments.

SatsPath is not:

- A custodial wallet.
 - A new Bitcoin consensus protocol.
 - A Lightning replacement.
 - An Ark replacement.
 - A full wallet in the first prototype.
 - A system that creates private keys for other people.
-

4. Core Flow

Registered user

Sender enters:
rodrigo@satspath.dev

SatsPath resolves the alias.

The resolver returns:
SignedPaymentProfile

The sender verifies:

- Alias binding
- Profile signature
- Public key fingerprint
- Supported payment methods
- Key age
- Expiration
- Recent key changes

The router evaluates:

- Amount
- Lightning availability
- On-chain fee rate
- Ark availability
- User preference
- Privacy policy
- Maximum fee

The router selects:
Lightning / On-chain / Ark

The sender confirms:
Pay

The payment is executed or simulated.

Unregistered user

Sender enters:
julian@example.com

SatsPath checks registry.

No profile is found.

SatsPath does not create keys for Julian.

Instead, SatsPath creates an invite / claim flow.

Julian receives a claim link.

Julian opens the link.

Julian generates keys locally.

Julian publishes a signed payment profile.

The sender resolves Julian's new profile.

The sender confirms payment.

Payment is sent to Julian's own keys.

Important rule:

The sender must never generate private keys for the receiver.

That would break self-custody.

5. Identity Model

SatsPath uses human-readable aliases, but the real cryptographic identity is a public key.

Example:

Alias:
rodrigo@satspath.dev

Identity public key:
02ab...ff12

```
Fingerprint:  
f8a2c1d9
```

The alias is for humans.

The public key is for cryptographic verification.

The profile signature proves that the owner of the identity key authorized the payment profile.

6. Alias Uniqueness

Names like this are not globally unique:

```
Rodrigo  
Alice  
Julian
```

A proper alias should include a namespace:

```
rodrigo@satspath.dev  
alice@example.com  
julian@bitcoin.dev
```

This allows SatsPath to support future resolver backends:

- Local registry
- DNS
- BIP-353
- Nostr
- Web-of-trust
- Self-hosted domains

For the MVP, use a local registry.

For production, support BIP-353 and/or DNSSEC-backed resolution.

7. Compatibility with Existing Standards

SatsPath should not compete against existing standards.

It should build above them.

BIP-353

BIP-353 resolves human-readable Bitcoin names through DNS TXT records with DNSSEC validation.

SatsPath can use BIP-353 as one resolver backend.

Example future flow:

```
ðrodrigo@satspath.dev
  ↓
DNSSEC-validated TXT record
  ↓
bitcoin: URI / payment instructions
  ↓
SatsPath payment capabilities
  ↓
Router
```

BIP-21 / BIP-321

Bitcoin URIs already exist for QR-based payment requests.

SatsPath should support its own URI format for the prototype, while remaining compatible with Bitcoin payment URIs later.

LNURL / Lightning Address

Lightning Address already allows email-like Lightning receiving.

Example:

```
rodrigo@getalby.com
```

SatsPath should treat Lightning Address as one possible payment method inside a broader profile.

Ark

Ark can be added as a payment method using an adapter interface first.

Do not attempt full Ark implementation in the first version.

8. SatsPath URI Format

SatsPath supports two URI forms.

Simple alias URI

```
satspath:rodrigo@satspath.dev
```

This means:

```
Resolve this alias and let the router decide.
```

Encoded payment request URI

```
satspath:v1:<base64url_json>
```

Decoded payload:

```
{
  "version": 1,
  "alias": "rodrigo@satspath.dev",
  "amount_sats": 21000,
  "memo": "Coffee payment",
  "expires_at": 1760000000,
  "profile_hint": null
}
```

The QR can encode this URI.

The QR should not need to change every time the user changes wallets.

The alias points to an updated signed profile.

9. Universal QR

The universal QR represents the payment identity or payment request.

Static QR

```
satspath:rodrigo@satspath.dev
```

Use case:

- Donation page
- Business counter
- Personal profile

- Reusable payment identity

Dynamic QR

```
satspath:v1:<encoded_payload>
```

Use case:

- Invoice
- Checkout
- Exact amount
- Specific memo
- Expiration time

The dynamic QR includes amount and memo.

The static QR only includes the recipient identity.

10. Payment Profile

A Payment Profile describes how the user can receive payments.

```
{
  "version": 1,
  "alias": "rodrigo@satspath.dev",
  "identity_pubkey": "02ab...",
  "methods": [
    {
      "type": "lightning",
      "label": "Main Lightning Address",
      "lightning_address": "rodrigo@getalby.com",
      "lnurl": null,
      "bolt12": null
    },
    {
      "type": "onchain",
      "label": "On-chain receive 001",
      "address": "bc1q...",
      "pubkey_hint": "03cd..."
    },
    {
      "type": "ark",
      "label": "Ark receiver",
      "server": "https://ark.example.com",
      "pubkey": "02ef..."
    }
  ]
}
```



```
"preferences": ["lightning", "ark", "onchain"],
"updated_at": 1760000000,
"expires_at": 1760600000,
"nonce": "random-128-bit-value"
}
```

11. Signed Payment Profile

A signed profile wraps the payment profile with a signature.

```
{
  "profile": {
    "version": 1,
    "alias": "rodrigo@satspath.dev",
    "identity_pubkey": "02ab...",
    "methods": [],
    "updated_at": 1760000000,
    "expires_at": 1760600000,
    "nonce": "random-128-bit-value"
  },
  "signature": "..."
}
```

The signature proves:

The identity private key authorized this payment profile.

The profile should be rejected if:

- Signature is invalid.
- Profile is expired.
- Alias does not match the requested alias.
- Identity key changed recently without warning.
- The resolver response is unsigned or malformed.
- The profile version is unsupported.

12. Cryptographic Model

Important correction

Public keys do not decrypt secrets.

Public keys are used to:

- Identify users.
- Verify signatures.
- Derive shared secrets in specific encryption protocols.
- Bind aliases to payment profiles.

For the MVP, the payment profile can be public and signed.

If private profile fields are needed later, use authenticated encryption with a proper ECDH-based protocol.

Recommended cryptography

Use:

- secp256k1 keys
- Schnorr signatures if possible
- Domain-separated message hashing
- Canonical JSON serialization
- Strong randomness from the operating system
- Local-only private key storage

Do not sign raw JSON directly unless the JSON serialization is deterministic.

The correct signing flow:

```
PaymentProfile
  ↓
Canonical JSON
  ↓
Domain-separated hash
  ↓
secp256k1 signature
  ↓
SignedPaymentProfile
```

Example domain separator:

```
SatsPathProfileV1
```

Pseudo-code:

```
let canonical = canonical_json(profile);
let message = sha256("SatsPathProfileV1" || canonical);
let signature = secp256k1_sign(message, secret_key);
```

Verification:

```
let canonical = canonical_json(profile);
let message = sha256("SatsPathProfileV1" || canonical);
verify(signature, message, profile.identity_pubkey);
```

13. Key Management

Identity key

The identity key signs the profile.

It should not receive payments directly.

```
Identity key = profile signing key
Payment keys = LN / on-chain / Ark keys
```

Do not mix identity keys and payment keys.

On-chain privacy

For on-chain payments, do not reuse addresses.

The profile should support multiple on-chain receive methods.

In production, the wallet should generate fresh addresses or use a reusable privacy-preserving method such as Silent Payments.

For the MVP:

```
{
  "type": "onchain",
  "label": "onchain-demo-001",
  "address": "bc1q...",
  "pubkey_hint": "03..."
}
```

Do not claim perfect privacy in the MVP.

Say:

SatsPath supports multiple on-chain keys/addresses to reduce address reuse.

Do not say:

SatsPath guarantees on-chain privacy.

14. Registry / Resolver

For the hackathon prototype, use a local registry.

.satspath/registry.json

Example:

```
{
  "rodrigo@satspath.dev": {
    "profile": {},
    "signature": "...",
  }
}
```

Required functions:

```
register_profile(signed_profile)
resolve_alias(alias) -> SignedPaymentProfile
is_registered(alias) -> bool
```

Future resolver backends:

Local Registry
DNSSEC / BIP-353
Nostr
HTTP well-known endpoint
Web-of-trust registry

Resolver security rule:

The resolver is not trusted.
The signed profile is trusted only if the signature verifies.

15. Registered User Flow

Step 1: User creates profile

```
satspath register rodrigo@satspath.dev
```

The app:

1. Generates identity key locally.
2. Creates demo payment methods.
3. Canonicalizes the profile.
4. Signs the profile.
5. Stores the signed profile in the registry.

Step 2: Sender resolves alias

```
satspath show rodrigo@satspath.dev
```

The app:

1. Loads signed profile.
2. Verifies signature.
3. Displays fingerprint.
4. Displays payment methods.
5. Warns if profile is expired or recently changed.

Step 3: Sender requests quote

```
satspath quote rodrigo@satspath.dev 21000
```

The app:

1. Resolves the profile.
2. Verifies signature.
3. Checks methods.
4. Fetches on-chain fee rate.
5. Evaluates Lightning.
6. Evaluates Ark.
7. Selects best route.
8. Shows quote.

Step 4: Sender pays

```
satspath pay rodrigo@satspath.dev 21000
```

MVP behavior:

```
Resolving alias...  
Verifying signed profile...  
Checking payment rails...  
Selected route: Lightning  
Reason: Amount is small and Lightning is available.  
Payment status: simulated_success
```

16. Unregistered User Flow

This is critical.

If the receiver has not published a profile, the sender cannot safely send money to them yet.

Do not create private keys for the receiver.

Correct MVP flow

```
satspath pay julian@example.com 21000
```

Output:

```
julian@example.com is not registered on SatsPath.  
  
No payment was sent.  
  
Invite link:  
https://satspath.local/claim?alias_hash=<hash>&amount=21000  
  
The receiver must claim this payment by generating their own keys locally.
```

Claim flow

```
Julian opens invite link.  
Julian installs/opens SatsPath wallet.  
Julian generates keys locally.  
Julian creates payment profile.
```

```
Julian signs profile.  
Julian publishes profile.  
Sender resolves Julian's profile.  
Sender confirms or auto-executes payment.
```

Payment intent

For the MVP, create a payment intent, not a real payment.

```
{  
  "intent_id": "uuid",  
  "alias_hash": "sha256(julian@example.com)",  
  "amount_sats": 21000,  
  "status": "waiting_for_claim",  
  "created_at": 1760000000,  
  "expires_at": 1760086400  
}
```

The payment should remain in the sender's control until the receiver claims with their own keys.

This preserves self-custody.

17. Invite Security

Invite links are dangerous if designed badly.

Minimum protections:

- Do not include the raw email if not needed.
- Use alias hash.
- Add expiration.
- Show the claimed alias before payment.
- Require sender confirmation before final payment.
- Warn if the profile was created recently.
- Never embed private keys.
- Never embed seed phrases.
- Never auto-pay large amounts after claim.

Example invite object:

```
{  
  "version": 1,  
  "alias_hash": "sha256(alias)",  
  "amount_sats": 21000,  
  "created_at": 1760000000,  
  "expires_at": 1760086400,  
}
```

```
"claim_url": "https://satspath.local/claim?...",  
"warning": "Receiver must generate their own keys locally."  
}
```

18. Payment Rails

18.1 Lightning

Lightning should be preferred for small, fast payments when available.

Supported fields:

```
{  
  "type": "lightning",  
  "label": "Main Lightning",  
  "lightning_address": "rodrigo@getalby.com",  
  "lnurl": null,  
  "bolt12": null  
}
```

MVP behavior:

- Validate Lightning Address format.
- Detect LNURL availability.
- Optionally fetch metadata.
- Simulate payment.
- Do not require real Lightning payment execution.

Future behavior:

- Fetch LNURL-pay metadata.
- Validate min/max sendable amount.
- Request invoice.
- Estimate fees.
- Pay invoice from user's Lightning wallet.
- Support BOLT12 offers.

Lightning should be selected when:

```
amount_sats < lightning_threshold  
AND Lightning method exists  
AND amount is within LNURL/BOLT12 limits  
AND expected fees are acceptable
```

Default MVP threshold:

100,000 sats

18.2 On-chain

On-chain should be selected when:

- Payment amount is large.
- The user wants stronger settlement.
- Lightning is unavailable.
- Ark is unavailable.
- On-chain fees are acceptable.

Supported fields:

```
{
  "type": "onchain",
  "label": "On-chain receive 001",
  "address": "bc1q...",
  "pubkey_hint": "03..."
}
```

Privacy rule:

Never assume one static on-chain address is private.

MVP rule:

Support multiple on-chain addresses.
Prefer unused addresses when possible.

Production rule:

Use wallet-generated fresh addresses or Silent Payments.

18.3 Ark

Ark should be included as a payment method but implemented as an adapter first.

Supported fields:

```
{
  "type": "ark",
  "label": "Ark receiver",
  "server": "https://ark.example.com",
  "pubkey": "02..."
}
```

MVP Ark behavior:

- Detect if Ark method exists.
- Check if Ark server URL is present.
- Use a mock Ark client.
- Simulate payment intent.

Do not attempt complete Ark implementation in v0.1.

Ark should be selected when:

```
Lightning is unavailable or too expensive
AND on-chain fee is too high
AND Ark method exists
AND Ark server is reachable
```

19. Routing Engine

The router receives:

```
pub struct RouteRequest {
  pub alias: String,
  pub amount_sats: u64,
  pub signed_profile: SignedPaymentProfile,
  pub urgency: PaymentUrgency,
  pub max_fee_sats: Option<u64>,
  pub max_fee_percent: Option<f64>,
}
```

It returns:

```
pub struct RouteQuote {
  pub selected_method: PaymentMethod,
  pub reason: String,
  pub estimated_fee_sats: Option<u64>,
  pub estimated_confirmation: Option<String>,
}
```

```
pub warnings: Vec<String>,  
}
```

20. Routing Algorithm

MVP algorithm

1. Verify signed profile.
2. Reject expired profile.
3. Extract available methods.
4. If amount < 100,000 sats and Lightning exists:
 choose Lightning.
5. Else fetch on-chain fee rate.
6. If on-chain fee rate is acceptable and on-chain address exists:
 choose On-chain.
7. Else if Ark exists:
 choose Ark.
8. Else fail.

More precise algorithm

Input:

- amount_sats
- payment profile
- urgency
- max_fee_sats
- max_fee_percent
- user rail preference
- network fee estimate

Step 1:
Verify profile.

Step 2:
Build candidate routes:

- Lightning candidate
- On-chain candidate
- Ark candidate

Step 3:
Score each candidate:

- availability
- expected fee
- expected speed
- privacy
- reliability

- user preference

Step 4:

Reject candidates that violate hard constraints:

- no method available
- fee above max
- expired invoice/profile
- invalid address
- unsupported network
- unsafe key change

Step 5:

Choose the best remaining candidate.

Step 6:

Return quote with explanation.

21. Fee-Rate Algorithm

SatsPath must not blindly trust a single fee source.

Recommended fee sources:

1. User's own Bitcoin Core node, if available.
2. Public mempool API, if node is unavailable.
3. Safe fallback/mock value for demo.

Fee source priority

```
if Bitcoin Core is available:
    use estimatesmartfee / local mempool data
else if mempool.space API is available:
    use recommended fees
else:
    use fallback fee policy
```

Mempool fee response

Expected structure:

```
{
  "fastestFee": 12,
  "halfHourFee": 10,
  "hourFee": 8,
  "economyFee": 5,
```

```
"minimumFee": 1
}
```

Urgency mapping

```
Urgent:
    target = fastestFee

Commercial:
    target = halfHourFee

Normal:
    target = hourFee

Economy:
    target = economyFee
```

Safety margin

Use a small safety margin to avoid underpaying.

```
selected_feerate = max(api_feerate, minimumFee)
selected_feerate = selected_feerate + ceil(selected_feerate * 0.10)
selected_feerate = max(selected_feerate, 1 sat/vB)
```

For MVP:

```
safety_margin = 10%
```

Estimated on-chain fee

The wallet should eventually calculate the real transaction virtual size from the PSBT.

For the MVP, use estimates:

```
P2WPKH 1 input / 2 outputs ≈ 140 vB
P2TR 1 input / 2 outputs ≈ 110 vB
```

MVP formula:

```
estimated_fee_sats = selected_feerate_sat_vb * estimated_vbytes
```

Example:

```
selected_feerate = 8 sat/vB
estimated_vbytes = 140 vB

estimated_fee = 8 * 140 = 1,120 sats
```

Fee acceptability

Define hard limits:

```
max_fee_sats = user configured limit
max_fee_percent = user configured percent of payment
```

Default MVP policy:

```
max_fee_percent = 2.0%
max_fee_sats = 5,000 sats
```

The fee is acceptable if:

```
estimated_fee_sats <= max_fee_sats
AND
estimated_fee_sats <= amount_sats * max_fee_percent
```

If the fee is too high:

```
Do not choose on-chain unless user explicitly confirms.
Try Lightning or Ark instead.
```

22. Route Scoring

Each route can receive a score.

Example:

```
score = speed_score + fee_score + privacy_score + reliability_score +
preference_score
```

For MVP, keep it simple.

Lightning score

Good when:

- Small payment.
- LNURL or Lightning Address exists.
- Invoice can be generated.
- Fee is low.
- Fast confirmation is desired.

Bad when:

- Amount is too large.
- Recipient has no Lightning method.
- Invoice generation fails.
- Liquidity/routing fails.

On-chain score

Good when:

- Amount is large.
- Fees are low.
- User wants final settlement.
- Recipient has an on-chain address.

Bad when:

- Fees are high.
- Amount is small.
- Address has been reused.
- User needs instant payment.

Ark score

Good when:

- Lightning is unavailable.
- On-chain fee is high.
- Recipient supports Ark.
- Ark server is reachable.

Bad when:

- Ark server is unavailable.
- Ark support is mock-only.
- User does not accept Ark trust model.

23. Suggested MVP Route Policy

```
if has_lightning && amount_sats <= 100_000 {  
    return Lightning;  
}  
  
let fee = get_onchain_fee_rate();  
  
if has_onchain && fee.hour_fee <= 10 {  
    return Onchain;  
}  
  
if has_ark {  
    return Ark;  
}  
  
if has_onchain {  
    return OnchainWithWarning;  
}  
  
return NoRouteAvailable;
```

Return explanations, not only results.

Example:

```
Selected route: Lightning  
Reason: Amount is below 100,000 sats and recipient supports Lightning  
Address.  
Estimated fee: unknown / wallet-dependent  
Expected settlement: instant
```

Example:

```
Selected route: On-chain  
Reason: On-chain fee rate is low at 5 sat/vB and payment amount is large.  
Estimated fee: 700 sats  
Expected settlement: ~1 hour
```

Example:

```
Selected route: Ark  
Reason: On-chain fee rate is high and Lightning is unavailable. Recipient  
supports Ark.
```


Estimated fee: server-dependent
Expected settlement: instant/off-chain

24. Split Payments

Split payments are not required for v0.1, but the protocol should be designed to support them later.

Future payload:

```
{
  "version": 1,
  "amount_sats": 100000,
  "splits": [
    {
      "alias": "merchant@satspath.dev",
      "percent": 70
    },
    {
      "alias": "supplier@satspath.dev",
      "percent": 20
    },
    {
      "alias": "platform@satspath.dev",
      "percent": 10
    }
  ]
}
```

Future router behavior:

Resolve every recipient.
Verify every profile.
Choose rail per recipient.
Execute or prepare payment batch.
Return result per split.

For v0.1:

Do not implement real split payments.
Only design the data structure.

25. Security Model

SatsPath should follow this security principle:

Trust cryptographic signatures, not servers.

What the server/registry can do

The server can:

- Store signed profiles.
- Return signed profiles.
- Store invite intents.
- Help discover aliases.

The server cannot:

- Spend user funds.
- Create receiver keys.
- Modify a profile without invalidating the signature.
- Prove identity by itself.

What the user controls

The user controls:

- Identity private key.
- Wallet private keys.
- Payment methods.
- Profile updates.
- Payment confirmation.

26. Threat Model Summary

Threat	Risk	MVP Mitigation	Future Mitigation
Fake alias registration	High	Mark aliases as local/unverified	DNSSEC, BIP-353, Nostr verification, web-of-trust
Server tampering	High	Signed profiles	Transparency log, multiple resolvers
Key replacement attack	High	Show key fingerprint and warnings	Key history, signed rotations, recovery keys
Email takeover	High	Email is not treated as cryptographic identity	DNSSEC, Nostr, hardware keys

Threat	Risk	MVP Mitigation	Future Mitigation
LNURL spoofing	Medium	Verify profile signature before using LNURL	HTTPS validation, domain checks, allowlist
On-chain address reuse	Medium	Support multiple on-chain methods	Fresh addresses, Silent Payments
Lost identity key	Medium	Warn that profile updates require key	Recovery policy, multisig identity
Malicious invite link	High	Expiring invite, alias hash, sender confirmation	Signed invites, domain verification
Ark server assumptions	Medium	Treat Ark as explicit payment rail	Server reputation, proof validation, exits
Public registry privacy leak	Medium	Store minimum public profile	Encrypted profile fields, private discovery

27. Key Rotation

Users must be able to rotate identity keys safely.

Bad rotation:

```
New key appears suddenly.
Wallet accepts it silently.
```

Good rotation:

```
Old key signs new key.
New key signs new profile.
Wallet shows rotation history.
```

Rotation object:

```
{
  "old_identity_pubkey": "02old...",
  "new_identity_pubkey": "02new...",
  "rotated_at": 1760000000,
  "old_key_signature": "...",
  "new_key_signature": "..."
}
```

MVP:

Show warning if identity key changed.

Production:

Require signed key rotation chain.

28. Profile Expiration

Profiles should expire.

Example:

```
{
  "updated_at": 1760000000,
  "expires_at": 1760600000
}
```

Why:

- Prevent stale payment data.
- Reduce risk from compromised old data.
- Force periodic refresh.

MVP default:

```
expires_at = updated_at + 30 days
```

If expired:

Reject profile or show strong warning.

29. CLI Commands

The prototype should expose these commands:

```
satspath init
satspath register <alias>
satspath show <alias>
satspath encode <alias> <amount_sats> --memo "..."
```

```
satspath decode <uri>
satspath quote <alias> <amount_sats>
satspath pay <alias> <amount_sats>
satspath invite <alias> <amount_sats>
satspath demo
```

Example demo

```
satspath init
satspath register rodrigo@satspath.dev
satspath show rodrigo@satspath.dev
satspath encode rodrigo@satspath.dev 21000 --memo "coffee"
satspath quote rodrigo@satspath.dev 21000
satspath pay rodrigo@satspath.dev 21000
satspath invite julian@example.com 21000
```

30. Project Structure

```
satspath/
├─ Cargo.toml
├─ README.md
├─ .gitignore
├─ crates/
│   └─ satspath-core/
│       ├── Cargo.toml
│       └─ src/
│           ├── lib.rs
│           ├── profile.rs
│           ├── crypto.rs
│           ├── codec.rs
│           ├── registry.rs
│           └─ errors.rs
│   └─ satspath-router/
│       ├── Cargo.toml
│       └─ src/
│           ├── lib.rs
│           ├── router.rs
│           ├── fees.rs
│           ├── lightning.rs
│           ├── onchain.rs
│           └─ ark.rs
│   └─ satspath-cli/
│       ├── Cargo.toml
│       └─ src/
│           └─ main.rs
```

```
├─ examples/
│   └─ rodrigo_profile.json
│   └─ demo_flow.md
└─ docs/
    └─ architecture.md
    └─ protocol.md
    └─ threat_model.md
```

31. Rust Modules

satspath-core

Responsible for:

- PaymentProfile
- SignedPaymentProfile
- PaymentMethod
- Signature verification
- URI encoding/decoding
- Local registry
- Invite objects

satspath-router

Responsible for:

- RouteRequest
- RouteQuote
- Fee estimation
- Lightning detection
- On-chain detection
- Ark mock client
- Routing decisions

satspath-cli

Responsible for:

- Demo commands
 - Human-readable output
 - Local testing
 - Hackathon presentation flow
-

32. Minimum Tests

Add tests for:

- Profile serialization
- Profile canonicalization
- Profile signing
- Valid signature verification
- Invalid signature rejection
- URI encoding
- URI decoding
- Alias registration
- Alias resolution
- Lightning route selection
- On-chain route selection when fees are low
- Ark fallback when on-chain fees are high
- Invite generation for unregistered alias

33. Product Style

SatsPath should feel like:

Simple
Trustworthy
Technical underneath
Calm on the surface
Premium but not flashy

Avoid crypto casino aesthetics.

Use:

- White or warm neutral background
- Black typography
- Bitcoin orange accent
- Route cards
- Verification badges
- Clear warnings
- Minimal motion
- Fee transparency

UI hierarchy:

1. Who are you paying?
2. How much?
3. Verified recipient?

4. Selected route
5. Fee estimate
6. Confirm

Example payment screen:

```
Pay Rodrigo
rodrigo@satspath.dev

Verified profile: yes
Identity fingerprint: f8a2c1d9

Amount: 21,000 sats

Selected route:
Lightning

Reason:
Fastest and cheapest route for this amount.

Estimated fee:
Low / wallet-dependent

[Confirm Payment]
```

For unregistered user:

```
Julian is not registered yet.

No payment has been sent.

Create an invite so Julian can claim this payment with his own wallet.

[Create Invite]
```

34. Hackathon Scope

Must build

- Rust workspace
- Payment profile
- Profile signatures
- URI encoder/decoder
- Local registry
- Registered user flow
- Unregistered user invite flow

- Fee-rate client
- Router
- CLI demo
- README
- Threat model

Should build

- Basic web UI
- QR display
- Route visualization
- Mock payment success

Do not build yet

- Full mobile wallet
- Real Ark implementation
- Real Lightning payment execution
- Real on-chain broadcasting
- Custodial escrow
- Automatic receiver key generation
- Complex split payments

35. Final Pitch

SatsPath is a universal Bitcoin payment resolver and router.

It allows a user to pay using one human-readable identifier while the protocol resolves a signed payment profile and automatically selects the best available rail: Lightning, on-chain, Ark, or future payment methods.

For registered users, SatsPath verifies their signed profile and routes the payment.

For unregistered users, SatsPath creates a claim flow where the receiver generates their own keys locally before any payment is sent.

The system keeps keys with the user, avoids custody, and builds on existing Bitcoin standards instead of replacing them.

36. Brutal MVP Truth

The first version should not claim:

We built a new Bitcoin payment standard.

It should claim:

We built a working prototype of a signed Bitcoin payment resolver and routing engine.

That is strong, credible, and possible in one day.

37. Disclaimer

This is hackathon software.

Do not use with real funds.

Do not treat the local registry as secure production infrastructure.

Do not send real payments until wallet integrations, resolver security, key rotation, and transaction verification are fully implemented and audited.